Roadmap

Near-term priorities, future decisions, and next development milestones.

Working draft

Near-term priorities, future decisions, and next development milestones.

Stabilization And Surface Polish

Current baseline

The repo already has:

- the core / runtime / app / src/app split
- packet schema and builder foundations
- SQLite-backed packet and revision storage
- cryptographic identity continuity
- packet-backed discussions and attestations
- a functional Nexus shell
- a Packet Explorer with live Search, Import, and Export workbenches

The next planning focus is still catch-up and semantic stabilization.

Near-term priorities

- keep identity/auth and discussions as the strongest reference verticals
- harden roles, trust, library, and votes around truthful current behavior
- harden Packet Explorer around truthful import, verification, and trust reporting rather than treating its current live workbenches as the main unfinished problem
- improve surface honesty through warnings, disabled-placeholder clarity, and route-state cleanup where needed
- treat recent dashboard, focus, and shared packet-card polish as good substrate work that should stay secondary unless a regression appears

Packet Explorer follow-up backlog

- richer provenance and peer-trust reporting on top of the new verification corridor

- stale-versus-current verification surfacing and revalidation affordances beyond the first truthful revision anchoring pass
- broader import history filtering, bundle lineage inspection, and verification follow-up entrypoints
- broader report semantics beyond verification and import
- Fork, Adapt, and Follow semantics
- diff views and revision compare
- richer link grouping and filtering
- eventual action execution through the trusted mutation pipeline rather than route-local writes
- first-class Bundle packets after verification and locality UX hardening, with legacy bundle JSON remaining a runtime transport format rather than becoming a schema-compatibility type

Working order

1. harden import verification and truthful packet-status reporting 2. improve locality creation UX on top of the new writer path 3. add provider-neutral locality standardization and duplicate or equivalence handling 4. tighten trust, role, and relationship modeling 5. only then widen into deeper policy-heavy and governance-heavy workflows

The dedicated validation workflow screen remains intentionally unsupported. The next major implementation chapter after the current verification tidy work is still locality UX, not first-class Bundle implementation.

Bundle direction already locked in

- Bundle should become a first-class packet type later rather than staying permanently as a runtime-only JSON wrapper.
- Rebundling or forwarding someone else’s bundle should create a new bundle packet, not a revision of the original bundle packet.
- Bundle history should therefore behave as a lineage graph of related bundle packets rather than one shared revision chain.
- Runtime importreport, exportreport, and verification_report packets should remain the portable record surface around bundle movement and validation.
- Local trust should stay node-local even when those reports travel with exported artifacts.

Initiatives, Locality, And Subscriptions

Initiative direction

Initiatives should be treated as generic Nexus policy and template lineages rather than OWA-specific hardcoding.

Working direction:

- OWA is one initiative inside Nexus
- assemblies can subscribe to recognized OWA policies, dependencies, templates, and packet sets while remaining valid Nexus objects if they fork
- official versus unofficial initiative lineage should be inspectable, not mystical

Initiative backlog

- initiative versions and subscription or update behavior, using Relation(subtype: subscription) as the canonical adoption/sync edge
- current official versus older official version support
- subscription alignment projection that compares inherited policy/dependency requirements against deselected or locally overridden refs
- official versus unofficial visibility modes
- feed, search, Library, and Explorer filtering by initiative and version
- actor-level or client-level preferences for what initiative visibility modes to show

Locality and scope backlog

- continue refining the Global + You baseline
- keep residence distinct from association
- keep canonical packet-native home locality relation writes and explicit compatibility projections as the base for later shell UI graph work
- keep packet-native follow and association relations as the base for the later shell graph UI pass
- treat the current ancestry and provisional Location(region) writer path as the new locality substrate rather than as pending exploratory work
- locality UX pass 1 is now the active baseline: guided search, search-to-create handoff, non-mutating review, actionable duplicate warnings, explicit home-locality toggle, and a preview-only home-branch checklist are all live
- locality foundations phase 2A is now the active next layer underneath that UI: descriptor-first locality rows, Unicode-safe normalization, sparse ordered ancestry, and descriptor storage in linked Location.spatial_payload

- the current runtime catch-up layer now sits between locality foundations and the later schema chapter: composite locality graph apply, centralized home or association or follow relation reads, temporary claimed-actor main visibility preferences, and server-projected shell sections are now the active bridge
- locality foundations phase 2B should then make home-tree inclusion and projection fields authoritative without redesigning the shell
- after that, continue locality standardization through provider-neutral candidate mapping, aliases, external refs, duplicate or equivalence handling, and broader non-US administrative structures
- preserve the distinction between mounted scopes, followed scopes, and merely known scopes
- follow the current runtime/projection catch-up pass with the schema chapter that packetizes preferences and broader relation semantics, then continue with the dedicated locality-standardization and provider pass rather than collapsing all three into one change set
- defer shared assembly custody or keyset work until after semantic foundations are clearer
- keep first-class Bundle packet work explicitly after the current verification and locality-UX chapters rather than letting runtime transport bundles quietly harden into a forever format

Trust, Policy, And Moderation

Trust and policy

Trust remains a major product-defining system.

Current direction:

- trust should stay inspectable and threshold-based
- roles should remain layered on scoped claims plus support or dispute evidence
- policies should define defaults where possible instead of hardcoding OWA behavior as universal law

Moderation and automoderation backlog

- generic automoderation support with element-defined policy baselines
- actor-level moderation preference controls
- explainable visibility projection rather than opaque hidden filtering
- soft visibility controls first, with harder runtime enforcement only where clearly needed

Packet-model questions to keep visible

- whether reactions should remain lightweight reaction-like signals or become a distinct type
- whether vouch and flag should become explicit reaction subtypes
- whether Claim and Reaction should stay split forever or eventually converge
- how recursive reaction graphs should be summarized safely in UI

These are active modeling questions, not current implementation commitments.

Governance And Execution

Governance direction

Governance should follow trust, policy, and inspectability rather than racing ahead of them.

Working direction:

- governance should follow verification truthfulness, trust and policy seams, locality and scope stabilization, and inspectable process rather than leapfrogging those foundations
- decisions begin as legitimacy records and civic signals
- proposals should declare what happens if they pass as explicitly and programmatically as possible
- some proposal shapes may later become effect-bearing through policy
- future resolutions and decisions should read as process-backed outcome artifacts rather than as magical authority events
- governance should resolve through the Trusted Regulation Coordinator and trusted planning/building pipeline rather than route-local vote logic

Policy resolver and voting foundation

Near-term governance work should establish the policy language and resolver before enabling live voting.

Milestones:

- formalize policy requirements for proposing, voting, attesting, reviewing, and moderating
- extend the initial Trusted Regulation Coordinator into the full policy resolver owner
- connect proposal creation to active governance policy
- connect vote eligibility to trust, role, association, and scope gates
- define quorum, threshold, vote method, and voting window semantics
- define decision creation rules and decision-report conditions

- keep automatic execution deferred until explicit built-in action handlers or canonical approval artifacts exist

Governance backlog

- real proposal drafting, editing, review, and publication flows
- real vote participation and result flows
- decision publication and linkage
- full policy resolver semantics on top of the initial Trusted Regulation Coordinator structure
- role-aware review checkpoints and trust-aware participation gates
- canonical approval packets or signed execution artifacts
- multi-key assemblies, custody transition, and decision-to-effect authority

Proposal and decision shape questions

- policy adoption proposals
- mission or coordination plan proposals
- resolutions
- signals or polls
- other explicit action packages

Automatic execution should remain future work and should be limited to explicitly supported built-in handlers or equivalent canonical approval artifacts.

Open Questions And Deferred Decisions

Near-term open questions

- How should initiative conformance and version recognition be represented?
- Which visibility modes should arrive first for official, unofficial, current-version, and older-version packets?
- How should Packet Explorer expose Fork, Adapt, Diff, Export, and Follow first?
- Where should moderation preferences live: actor policy, client preference, or both?
- What exact local trust state should be attached to imported or re-verified packets?
- Which packet relationships should a future first-class Bundle packet record directly: carried packet refs, root refs, lineage refs, verification refs, or all of the above?
- Which remaining shell/interface preferences should join main visibility and section-display options under the new packet-backed Preference.element model?

- Should defaults remain inside Definition parts, or should a future first-class Default packet exist once the current definition system proves the need?

Deeper design questions

- How expressive should the policy language become before it risks becoming a full rule engine?
- How much projection behavior belongs in packet definitions versus runtime projection profiles?
- What exact artifact gets signed in the final flow: user intent, packet candidate, finished packet bundle, runtime certification report, or some combination?
- How should imported definition/profile packets be trusted, ignored, sandboxed, or adapted?
- Which future relationship workflows should remain claim-mediated, and which should be written directly as plain relations?
- What exact authority model should govern multi-key assemblies and custody transition?
- What exact event turns a decision from a signal into an effect-bearing authority artifact?
- Which built-in action handlers, if any, should Nexus eventually support for automatic execution?
- How broad should the new Report type become beyond verificationreport and importreport?
- How should recorded verification or import reports, local verification caches, and later live read-model verification results relate over time?
- Should future node-to-node trust let one node weigh validation reports signed by another node, and if so through what policy seam?
- When, if ever, should packet-based compatibility manifests complement the current code-based adapter registry?
- How much default flattening should future rebundling do when one bundle carries packets that already arrived through an older bundle lineage?

Explicitly unsupported

- another broad architecture rewrite
- Discord adapter revival
- mesh transport and federation work
- real-time chat
- advanced reputation math
- deep governance automation before the trust and policy seams are clearer
- a dedicated validation workflow screen before locality UX and geo standardization are settled

- packet-based compatibility manifests as active implementation rather than future design direction
- first-class Bundle implementation before locality UX and geo standardization are settled
- authoritative selective home-tree storage and projection semantics before the current locality descriptor, Unicode, and sparse-path foundations are settled
- provider-backed locality candidate ingestion, geometry, and external locality APIs before the current descriptor and Unicode foundation pass is absorbed
- expanding the legacy runtime preference cache for new behavior instead of extending the signed packet-backed Preference.element corridor

Bundle direction now decided

- Bundle should become a first-class packet type later.
- Rebundling or forwarding another node’s bundle should create a new bundle packet rather than revising the older bundle packet.
- Bundle history should be modeled as a lineage graph, not one shared revision chain.
- Legacy bundle JSON artifacts should remain runtime import/export adapters until that packet type exists.